
Rapport de Projet

Développement Web 2 - L3 Informatique

Éditeur de texte collaboratif en temps réel

GIRAUD NICOLAS
COURT ANTHONY
CONTANT THIBAUT

18 mai 2026

Table des matières

1	Introduction	1
2	Description de l'application et fonctionnalités	1
2.1	Gestion des utilisateurs et authentification	1
2.2	Gestion des documents et des droits d'accès	1
2.3	Édition collaborative et communication	2
3	Conception et Architecture	2
3.1	Diagramme UML	2
3.2	Modèle Relationnel	4
4	Bilan des implémentations	4
4.1	Points forts	4
4.2	Limites et points d'améliorations	5
5	Ressources Extérieures Utilisées	5
6	Organisation du Travail	5
7	Conclusion personnelle	7

1 Introduction

Dans le cadre de l'enseignement "Développement Web 2" du semestre 6, nous avons choisi de concevoir une application web interactive.

Notre choix s'est porté sur la réalisation d'un **éditeur de texte collaboratif en temps réel**. Cette application permet à plusieurs utilisateurs de se connecter, de créer des documents, de gérer des droits d'accès avancés (propriétaire, éditeur, lecteur) et de modifier simultanément un même fichier tout en communiquant via une messagerie instantanée intégrée.

Ce rapport présente en détail la conception de notre application.

2 Description de l'application et fonctionnalités

L'application que nous avons développée est un environnement d'édition de texte collaboratif en ligne. Son objectif principal est de permettre à plusieurs utilisateurs de gérer et d'éditer des documents de manière sécurisée et interactive. Notre application propose une gestion des droits d'accès et permet une communication en temps réel grâce à l'utilisation des WebSockets.

2.1 Gestion des utilisateurs et authentification

Lorsque l'on arrive sur la page d'accueil du site, les utilisateurs doivent s'inscrire puis se connecter pour accéder à la plateforme. Les administrateurs du site disposent de privilèges supérieurs dès leur connexion, leur permettant de modérer la plateforme en ayant la possibilité de supprimer des utilisateurs ou des documents.

2.2 Gestion des documents et des droits d'accès

Une fois authentifié, l'utilisateur accède au tableau de bord principal (la page DocumentChoice). L'application permet d'y créer un nouveau document – ce qui génère automatiquement un fichier sur le serveur – ou d'en rechercher

un existant. LChaque document est régi par une table de permissions stricte offrant trois niveaux d'implication :

OWNER (Propriétaire) : Ce rôle est attribué par défaut au créateur du document. Le propriétaire détient les droits absolus sur son fichier. Il peut l'éditer, inviter d'autres utilisateurs en leur octroyant des droits d'écriture, bannir des lecteurs, ou encore supprimer définitivement le document du serveur.

WRITER (Éditeur) : Les utilisateurs invités avec ce rôle peuvent participer activement à la rédaction. Ils ont la capacité de modifier le document, de sauvegarder leurs modifications sur le serveur et de télécharger le fichier sur leur propre machine.

VIEWER (Lecteur) : Par défaut, le reste des membres du site se voit attribuer ce rôle sur un nouveau document public. Ils peuvent uniquement consulter le contenu sans avoir la possibilité de le modifier.

2.3 Édition collaborative et communication

Enfin, l'espace de travail (Editor) constitue le cœur du projet. Cette interface affiche la liste des collaborateurs ayant accès au document en cours et précise leur rôle. L'édition du texte est rendue dynamique grâce aux WebSockets qui permettent de gérer les modifications en temps réel. Nous avons aussi intégré un système de messagerie instantanée (Chat) permettant aux utilisateurs d'échanger en direct pendant qu'ils travaillent sur un même document.

3 Conception et Architecture

3.1 Diagramme UML

Voici le diagramme UML de notre projet
Ce diagramme a été réalisé grâce a UMLet

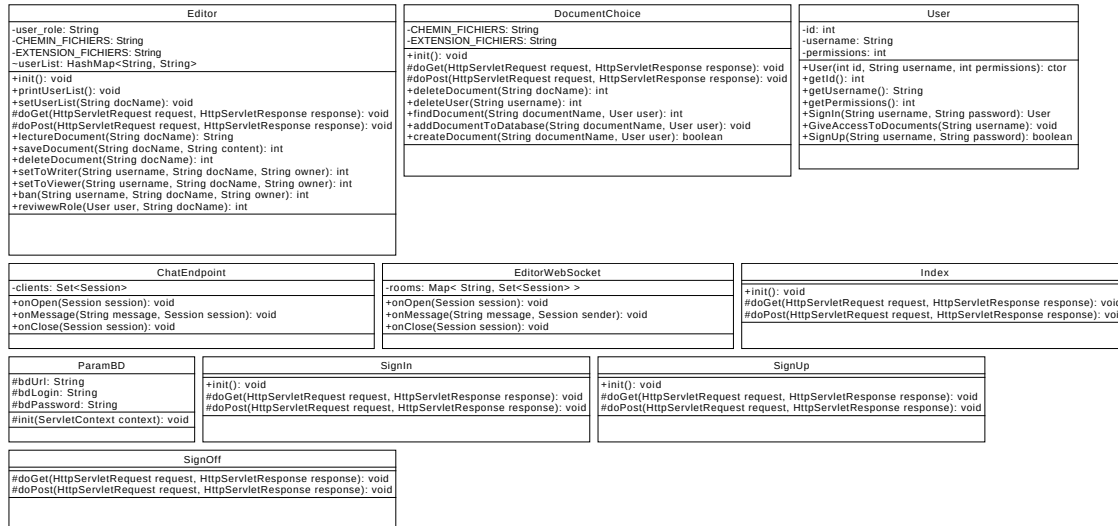


Figure 1: Diagrammes UML du projet - Page 1

Notre application possède plusieurs grandes fonctionnalités : D'abord, la gestion des utilisateurs : les classes SignIn, SignUp et SignOff gèrent la connexion, l'inscription et la déconnexion, en s'appuyant sur la classe User qui stocke les informations de chaque utilisateur (identifiant, nom, niveau de permission).

Ensuite, la gestion des documents : la classe DocumentChoice permet de créer, supprimer et rechercher des documents, tandis que la classe Editor se charge de leur lecture, sauvegarde et modification. L'éditeur gère également les droits d'accès, en permettant d'attribuer à un utilisateur un rôle de lecteur ou de rédacteur, voire de le bannir d'un document.

Enfin, la communication en temps réel est assurée par deux composants WebSocket. EditorWebSocket synchronise les modifications entre les utilisateurs travaillant simultanément sur un même document, et ChatEndpoint propose un système de messagerie intégré.

La classe ParamBD, quant à elle, centralise la configuration de la base de données utilisée par l'ensemble de l'application.

3.2 Modèle Relationnel

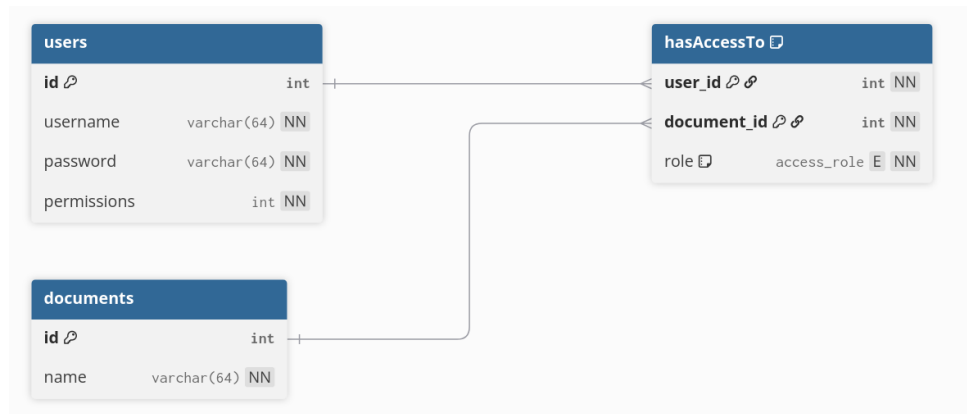


Figure 2: Diagramme Relationnel de la Base de Données (users, documents, hasAccessTo)

Notre base de données s'articule autour de trois entités principales. Les tables users et documents sont indépendantes et possèdent chacune un identifiant (clé primaire). La dernière table "hasAccessTo" permet de lier un utilisateur à un document. Cette table possède un attribut role (défini par une énumération), qui détermine le niveau de privilège de l'utilisateur sur le document.

4 Bilan des implémentations

4.1 Points forts

Nous avons réussi à développer les fonctionnalités principales de notre application. L'inscription et la connexion des utilisateurs fonctionnent parfaitement et sont sécurisées. La création de documents et leur enregistrement dans la base de données sont également opérationnels. Nous avons aussi ajouté un système de rôle : l'application différencie très bien les Propriétaires, les Éditeurs et les Lecteurs, et donne les bonnes permissions à chacun. Enfin, les administrateurs disposent bien des outils nécessaires pour supprimer des utilisateurs ou des documents en cas de besoin.

4.2 Limites et points d'améliorations

Le design de l'application (CSS) : L'interface est restée très basique et visuellement simple. On n'utilise pas JavaScript pour améliorer le design. Nous avons préféré passer notre temps à faire fonctionner la logique du site en arrière-plan plutôt qu'à décorer les pages.

Présence et Curseurs en temps réel L'éditeur ne permet pas de voir où les autres travaillent précisément. Nous aurions pu rajouter des curseurs à l'image de Google Docs, cela éviterait que deux personnes n'essaient de modifier la même phrase en même temps.

5 Ressources Extérieures Utilisées

Pour mener à bien ce projet, nous avons utilisés plusieurs outils et bibliothèques externes. Lors de la phase de conception, nous avons eu recours à UMLet pour réaliser notre diagramme UML, tandis que nous avons utilisé la plateforme en ligne dbdiagram.io pour concevoir notre modèle relationnel de base de données. Côté implémentation, notre architecture repose sur le pilote JDBC, placé dans notre dossier WEB-INF/lib. Pour la partie interface utilisateur, nous avons intégré la bibliothèque JSTL (Jakarta Standard Tag Library) à nos vues (.jsp). Cela nous a permis de respecter de respecter les normes de MVC en évitant de mélanger la logique métier et la vue. Le projet s'exécute au sein du serveur d'application Apache Tomcat et utilise les WebSockets pour assurer la communication en temps réel de l'éditeur et du chat.

6 Organisation du Travail

Nous avons tenté de répartir le travail de manière équitable.

Voici la répartition dans un tableau.

Nous avons passé environ 20h chacun sur ce projet

Membre	Rôles et Tâches principales	Temps (est.)
Nicolas Gi-raud	Développement intégral du module de discussion / Création du point d'accès serveur (ChatEndpoint.java) avec l'API WebSockets, et gestion de la connexion asynchrone côté client (chat.js) / Développement du serveur WebSocket pour l'éditeur (EditorWebSocket.java) / Développement de la Servlet DocumentChoice.java gérant la création de documents sur le serveur	~20h
Anthony Court	Développement complet des Servlets d'inscription (SignUp.java) et de connexion (SignIn.java) / Création des scripts SQL (creation.sql, insertion.sql), définition du modèle relationnel (les 3 tables dont hasAccessTo) et mise en place de la connexion JDBC via la classe ParamBD.java / Intégration du système de rôles croisés (OWNER, WRITER, VIEWER) entre les utilisateurs et les fichiers	~20h
Thibault Constant	Création des pages JSP (Index.jsp, DocumentChoice.jsp, Editor.jsp) avec intégration de JSTL pour dynamiser les vues / Conception de la charte graphique et du fichier style.css / Mise en place de l'interface de l'éditeur de texte et développement de la Servlet Editor.java / Développement du script JavaScript (editor.js)	~20h
Temps global consacré au projet		~60 h

Table 1: Répartition des tâches et estimation du temps de travail

7 Conclusion personnelle

Pour conclure, ce projet nous a permis de mettre en pratique les concepts essentiels vu en cours tels que la communication en temps réel grâce aux WebSocket ou encore la gestion des droits d'accès. Ce projet a considérablement renforcé notre maîtrise globale du modèle client-serveur, constituant ainsi une base technique solide pour nos futurs projets professionnels.